

-Aktuelle Sicherheitsthemen-

Linux Kernel Exploit CVE-2024-1086

Student:	Clara Stoeckel	93911
Universität:	Hochschule Karlsruhe – Technik und Wirtschaft	
Studiengang:	Informatik	
Semester:	Wintersemester 2025	
Dozent:	Prof. Dr. Patrick Baier	
Bearbeitet am:	25. November 2025	

Inhaltsverzeichnis

1	Abstract	1
2	Grundlagen	1
2.1	Der Linux Kernel & Privilege Model	1
2.2	Speicherverwaltung im Kernel	2
2.3	Netfilter & nf_tables	3
3	CVE-2024-1086	3
3.1	Verwundbare Komponente & Ursache	3
3.2	Exploitation	3
3.2.1	Heap Grooming & SLUB-Manipulation	3
3.2.2	Privilege Escalation zur Root-Shell	3
3.3	Patch & Mitigationen	3
4	Aktuelle Relevanz & Einordnung	3
	Literaturverzeichnis	4
	Abbildungsverzeichnis	5

1 Abstract

Diese Ausarbeitung analysiert die Use-After-Free-Schwachstelle CVE-2024-1086 im Netfilter-Subsystem `nf_tables` des Linux-Kernels, welche alle Kernel-Versionen von 5.14 bis 6.6 betrifft und mit einem CVSS-Score von 7.8 als hochkritisch eingestuft ist. Die Schwachstelle entsteht, weil die Funktion `nft_verdict_init()` positive Werte als Drop-Fehler im Hook-Urteil zulässt. Dies kann ein Double Free auslösen, wenn ein `NF_DROP` mit einem `NF_ACCEPT` ähnelnden Fehlercode ausgegeben wird [1]. Ein lokaler Angreifer kann diesen Fehler durch gezielte Heap-Manipulation ausnutzen, um vollständige root-Rechte zu erlangen.

Ausgehend von den Grundlagen des Linux-Speichermanagements und der `nf_tables`-Architektur wird der Fehler im Quellcode lokalisiert, der Exploit-Mechanismus nachvollzogen und der behebende Kernel-Patch analysiert. Abschließend werden bestehende Mitigationsmaßnahmen bewertet und ein Ausblick auf zukünftige Schutzmaßnahmen gegeben.

2 Grundlagen

2.1 Der Linux Kernel & Privilege Model

Der Linux Kernel bildet die Hauptkomponente des Linux Betriebssystem und dient als Schnittstelle zwischen Hardware und den Prozessen eines Computers. Hierbei hat er die folgenden vier grundlegende Funktionen: Speicherverwaltung, Prozessmanagement, Gerätetreiber und Systemaufrufe sowie Sicherheit. Er agiert im Hintergrund und ist für den Nutzer unsichtbar, solange er korrekt implementiert ist. [2]

Eine sogenannte Userspace-Anwendung, also eine Anwendung die der Nutzer sieht und mit ihr agiert, interagiert mit dem Kernel über eine Systemaufrufchnittstelle (System Call Interface (SCI)) [2]

Linux unterteilt die Berechtigungen des Root-Nutzers in separate Privilegien (Capabilities). Privilegien können eine Teilmenge der Zugriffskontrollregeln des Unix-System umgehen. Zum Beispiel die Berechtigungen eines Benutzers ändern. Dabei verfügt jeder Linux-Prozess über drei Arten von Berechtigungen

- Effective (Effektiv): Berechtigungssatz der Entscheidungen über Zugriffskontrolle trifft
- Permitted (Zulässig): Liste von Privilegien, die der Prozess im aktiven Zustand verwenden darf
- Inheritable (Vererbbar): Berechtigungssatz, welche Privilegien ein Prozess erwerben kann

[3]

Dies erklärt, wieso es fatal ist, wenn unberechtigte Personen Zugriff zu Rootrechten bekommen. Sie können dann alle möglichen Änderungen auf Betriebssystemebene vornehmen, Daten lesen und schreiben, Nutzungsrechte verändern und mehr.

2.2 Speicherverwaltung im Kernel

Der Kernel verwaltet seinen Speicher über die Funktionen `kmalloc()` und `kfree()`, sowie den SLUB-Allocator. Mittels `kmalloc()` kann Speicherplatz angefragt werden. Der SLUB-Allocator weist dann einen freien Slot in einem Speicher-Cache passender Größe zu. Objekte gleicher Größe werden dabei in gemeinsamen Pools, den sogenannten *Slab-Caches*, verwaltet **love2010linux**. `kfree()` gibt den Speicherplatz wieder frei, sodass der SLUB-Allocator ihn erneut zuweisen kann. Der freigegebene Speicher wird dabei nicht sofort gelöscht oder gesperrt, sondern lediglich als verfügbar markiert **kernel'memory**.

Ein *Use-After-Free* (UAF) entsteht, wenn Speicher freigegeben wurde, der Zeiger darauf jedoch weiterhin verwendet wird. Ein solcher Zeiger wird als *dangling pointer* bezeichnet **love2010linux**. Da der SLUB-Allocator freigegebene Slots umgehend für neue Allokationen gleicher Größe wiederverwenden kann, besteht die Möglichkeit, dass ein anderes Kernel-Objekt denselben Speicherbereich belegt. Schreibt der dangling pointer anschließend in diesen Bereich, werden die Felder des neuen Objekts unkontrolliert überschrieben.

Ein *Double Free* entsteht, wenn `kfree()` zweimal auf denselben Zeiger aufgerufen wird. Dabei wird die interne Verwaltungsstruktur des SLUB-Allocators korrumpiert, sodass derselbe Speicherbereich zwei verschiedenen Allokationen gleichzeitig zugewiesen werden kann **love2010linux**. Dies ist die direkte Ursache von CVE-2024-1086, wie in Abschnitt ?? beschrieben.

Heap Grooming bezeichnet die gezielte Manipulation des Kernel-Heaps, um einen dangling pointer auf ein vom Angreifer kontrolliertes Objekt zu lenken **hawkes2008**. Das Vorgehen erfolgt in drei Schritten: Zunächst wird durch gezielte Allokationen der Heap in einen vorhersehbaren Zustand gebracht. Anschließend wird das Zielobjekt freigegeben. Zuletzt allokiert der Angreifer ein eigenes Objekt gleicher Größe, welches den freigewordenen Slot übernimmt. Der dangling pointer zeigt nun auf das kontrollierte Objekt.

Ein bevorzugtes Angriffsziel ist die `cred`-Struktur des Linux-Kernels. Sie speichert die sicherheitsrelevanten Attribute eines Prozesses, darunter die User-ID (`uid`) und Group-ID (`gid`) **kernel'cred**. Gelingt es einem Angreifer, diese Felder auf den Wert 0 zu setzen, erhält der Prozess Root-Rechte. Genau dieses Prinzip liegt dem Exploit von CVE-2024-1086 zugrunde.

2.3 Netfilter & nf_tables

3 CVE-2024-1086

3.1 Verwundbare Komponente & Ursache

3.2 Exploitation

3.2.1 Heap Grooming & SLUB-Manipulation

3.2.2 Privilege Escalation zur Root-Shell

3.3 Patch & Mitigationen

4 Aktuelle Relevanz & Einordnung

Literaturverzeichnis

- [1] SUSE, *CVE-2024-1086 Common Vulnerabilities and Exposures*, <https://www.suse.com/security/cve/CVE-2024-1086.html>, Zugegriffen am: 02. April 2026, 2024.
- [2] RedHat, *Was ist der Linux-Kernel?* <https://www.redhat.com/de/topics/linux/what-is-the-linux-kernel>, Zugegriffen am: 08. April 2026, 2019.
- [3] PrivAnalyzer, *PrivAnalyzer: Measuring the Efficacy of Linux Privilege Use*, <https://ieeexplore.ieee.org/document/8809490>, Zugegriffen am: 21. April 2026, 2019.

Abbildungsverzeichnis